The tools and techniques of R programming
Lesson-4
R Programming with the ITERATION Logic

By
Prof.Dr. A.B.Chowdhury,HOD,CA

Techno India University, West Bengal,India
For any query,Email-Id:hod.ca@technoindiaeducation.com

September 18, 2021

Objective of this Lesson

The objective of this lesson is to illustrate the idea of repeated execution of a set of code till the user desires. The logic structure used for this purpose is called a **'loop'** structure. In the literature of systems, the structure is known as the **'Iteration logic'** structure. This logic structure is inevitable in computer programming, because most often we require the repeated execution of a part or whole of a set of programming statements until we meet our requirement.

Prerequisite: Knowledge of a programming language will be an added advantage.

The concept of LOOP or ITERATION structure

we shall now study the most important logic structure used in programs. This logic structure enables us to perform repetitive a task .Such a logic structure is popularly known as loop structure also. This logic structure is used when we require executing the same set of codes for a desirable number of times. The desirable number may also be kept indefinite depending on the user's choice during the execution. There are two forms of iteration, namely,

1. **Pretest iteration** and
2. **post-test iteration**

In case of **pretest iteration**, **a predicate**, commonly known as a condition or conditional expression, is checked first; if it evaluates to true, only then the set of statements under that structure is executed. In case of **post-test iteration**, a set of statements under the loop is executed first and then a conditional expression is checked to decide whether the repetition is to be done or not. Thus, **the statements in the post-test iteration are executed at least once. On the contrary, the statements under the pretest iteration may not be executed even once.**

Flowchart depicting Pretest iteration

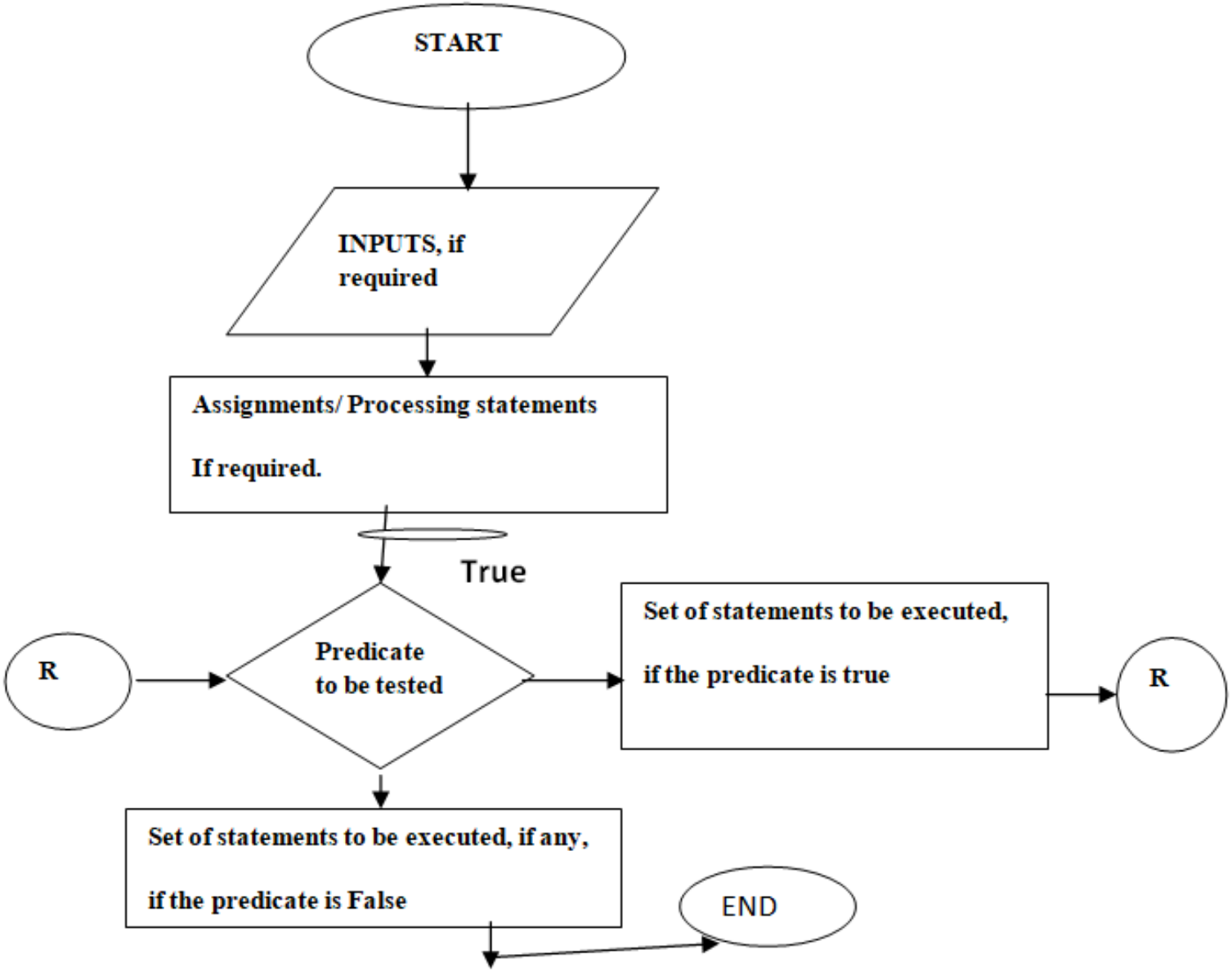

Figure: Pictorial presentation of 'Pretest iteration'

Flowchart depicting post-test iteration

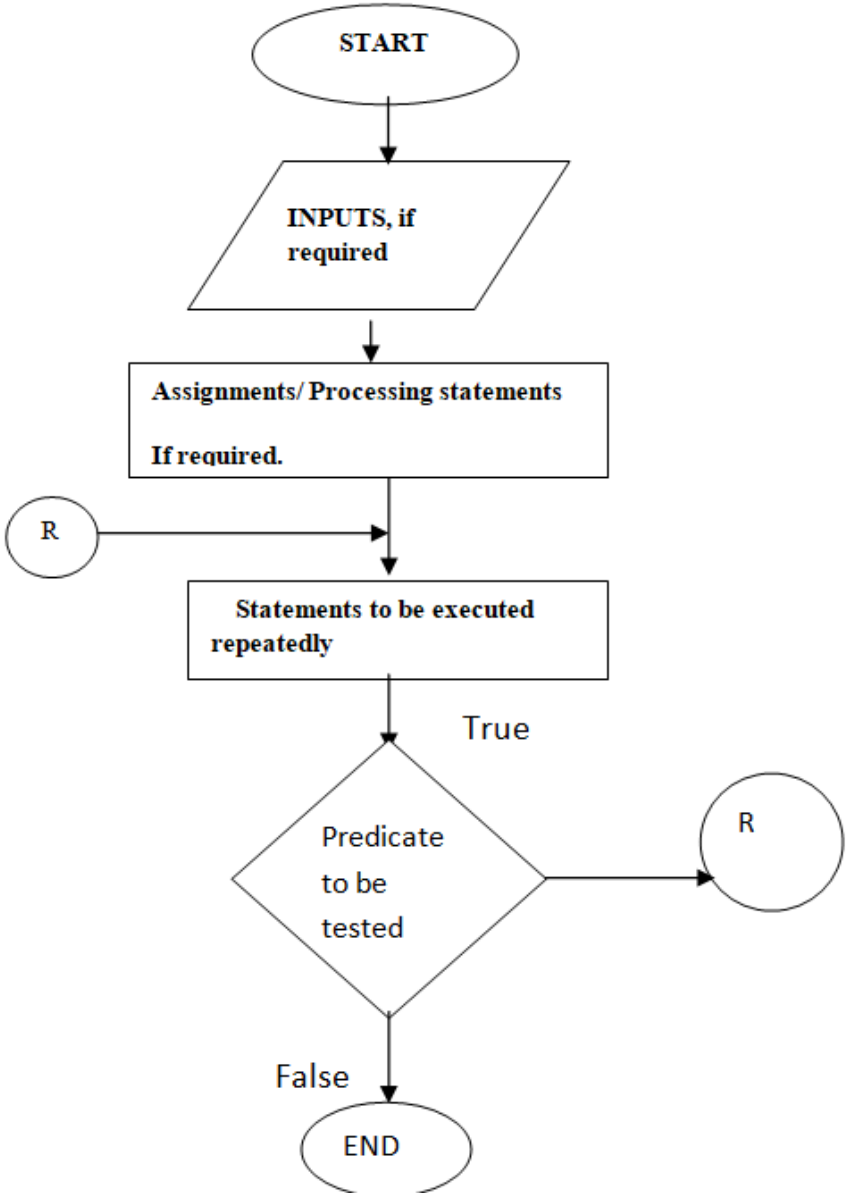

Figure: Pictorial presentation of 'post-test iteration'

THE ITERATION STRUCTURES SUPPORTED BY R

Pretest Iteration. R supports two structures for pre-test iteration as stated below:

1. **while(conditional-expression)** {**set of statements to be repeated** }
2. **for (expression)** {**set of statements to be repeated structure**}

The actual syntax of the loop structures are shown below:

The basic syntax for creating **a ' while' loop in R** is as shown below:

while (conditional-expression to be tested) { **Set of R statements to be executed**

}

Let us have some illustrated loop with this structure as under.

Count=1

while(Count<=5) {

print(" Welcome to R 'while' loop")

Count= Count+1

}

THE ITERATION STRUCTURES SUPPORTED BY R Contd.

If the preceding code is executed in R environment, then we shall have the following output:

[1] "Welcome to R 'while' loop"
[1] "Welcome to R 'while' loop"
[1] "Welcome to R 'while' loop"
[1] "Welcome to R 'while' loop"
[1] "Welcome to R 'while' loop"

Now, we shall study the **structure of the R for loop**. The basic syntax of this loop is as shown below:

for (variable in sequence) {
set of statements to be executed repeatedly
}

Here, **variable** refers to any R variable used for taking a set of values mentioned by the sequence one by one to execute the set of statements written for repeated execution.Here, the **increment of the loop variable** will always be by 1. However, sometimes, we need to increase the loop variable by any desired step size. For this purpose, the loop structure should be written as follows:

for(loop-variable in seq(beginning-value, ending value, step-size)){ **loop state-**

THE ITERATION STRUCTURES SUPPORTED BY R Contd.

Another point to be borne in mind is that **R does not compare the initial value of the loop variable with the ending value. It first executes the loop with the initial value, increments or decrements the value by the appropriate step value and then compares the resulting value with the ending value before going to execute the loop statements for the second time to ascertain whether to continue the loop or to discontinue.** The following code snippet illustrates this feature:

```
>for(i in seq(1,0,-2))print(i)
[1] 1
```

Now,let us have some illustration of execution of a for loop below:

```
 s=0
for(I in 1:10){
s=s+I
}
cat("The sum of the first 10 natural numbers is: ",s)
```

Here, the sequence is the set of natural numbers from 1 to 10 and I is any R variable.The output of the above code is:

```
[1] "The sum first 10 natural numbers is: 55"
```

The ‘repeat’ loop structure

There is no loop structure in R strictly following the definition of post-test iteration. However, there is a third loop structure in R that may be used in that format to some extent depending on the way how the user utilizes the loop structure. This loop structure is known as **repeat loop structure**. **Truly speaking, the repeat loop is a never ending loop i.e. an infinite loop.** But, there is a control statement in R that can be used to come out of the loop at any point of action. It is the **break** statement. This statement is usually written under a conditional statement to break the repetition. If this conditional statement is written just before the ending brace of the loop, then only we get the flavor of the post-test iteration. In general, the loop structure can be demonstrated by using the flowcharting symbols as shown in the figure below:

The repeat loop structure depicted in a flowchart

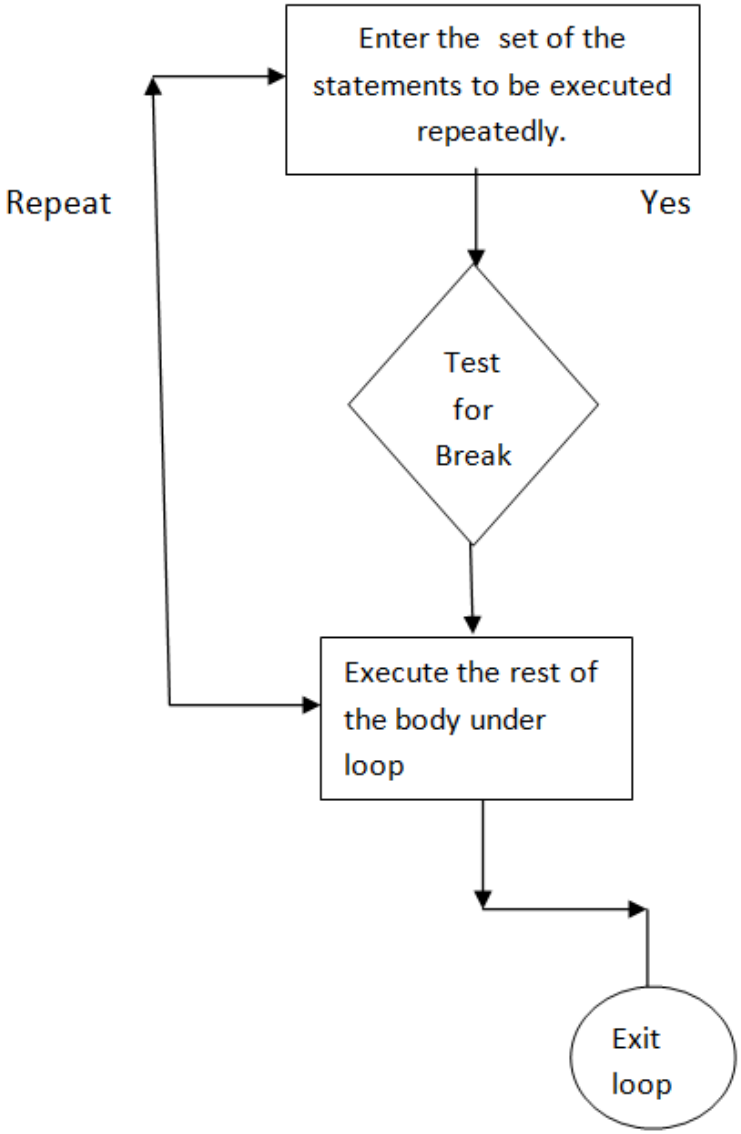

Figure: Pictorial presentation of 'repeat' loop structure

The repeat loop structure

In terms of R, the structure can be written as follows:

repeat set of statements to be executed repeatedly

There is another control statement in R that is often used in a for loop. It is the keyword: **next**. When the R system encounters the next command, it suspends execution of the remaining statements in the body of the loop , goes to the beginning of the loop and starts execution of the loop statements with the next value of the loop variable, if any.

Let us have an illustration of the repeat loop structure as in the R script below:

```
 count=1
>repeat{
+ print(" This is the repeat loop")
+ count=count+1
+ if (count>3)break
+ }
```

The output of this loop is shown in the figure below:

```
[1] " This is the repeat loop"
[1] " This is the repeat loop"
[1] " This is the repeat loop"
```

Illustrative example-4.1 using iteration

Example-4.1. It is required to obtain the sum of consecutive even numbers starting from 2 until the sum just exceeds 1000 and then to show the sum and the number of even numbers added.

Task Analysis: This is a very easy problem in which we shall not require any input value, because, the numbers to added can be generated through calculation. The solution in R code is given below:

```
# Program to determine the sum of even numbers.
n=2
s=0
cnt=0
while(s<=1000){
s=s+n
cnt=cnt+1
n=n+2}
cat("The desired sum of consecutive even numbers is: ",s)
cat("The number of even numbers added is: ",cnt)
print("End of Program.....Thank You")
```

Illustrative example-4.2 using iteration

Example-4.2. It is required to obtain the reversed form of a given number by using some R script.

Task Analysis: To obtain the reversed form of a given number, we proceed as explained below:We first obtain the rightmost digit of the number by dividing it by 10 and collect the remainder. This digit of the remainder is then stored in a variable and then the given number is replaced with its integer quotient and the division is repeated by treating quotient as the given number.The process is to be repeated until the integer quotient is reduced to zero.

To form the reversed form of the given number,we first define a variable with an initial value 0 before the start of the loop and update it after obtaining each remainder with the following formula:

variable=variable with the old value*10 + current remainder

However, the process described above will not show the zeroes at the end of the given number, if any. So, we have added some additional codes in R to concatenate zeroes at the leading positions as may be desired. These codes are underlined for the readers' perusal. We have used **paste() function** for the concatenation. The key-word **sep=""**,implies no spaces in the concatenated zeroes and **collapse=NULL** means the that the number is to be shown as a single string. This will give us the reversed number in the variable considered.

Illustrative example-4.2 in R using iteration

```
# Reversing a given number
cat("Enter the any whole number")
n=scan(n=1)
rn=0
cnt=0
while(n>0){
r=n%%10
if(r==0)
cnt=cnt+1
rn=rn*10+r
n=n%/%10  This is to obtain the integer part of the quotient.
}
if(cnt>0)
rn=as.character(rn)
while(cnt>0){
rn=paste('0',rn,sep="",collapse=NULL)
cnt=cnt-1
}
cat("The reversed number is: ",rn)
print("End of program.... Thank you")
```

Illustrative example-4.3 in R using iteration

Example-4.3. A program in R is required to be developed to show the divisors of a given number.

Task Analysis: This is also a very easy task. A number is said to be divisible by another number if the division of the former number by the latter number leaves zero remainder. We shall have to think of the solution in the way a child will think of it. We shall start checking the divisibility starting from one (1) and continue the checking until we reach the half of the number, because, after the half of the number, only the number itself can be the divisor. So, we should not check beyond the half value to avoid unnecessary calculation. The required codes in R have been shown in the figure below:

```
# Program to determine the divisors of a given number
cat("Enter any whole number")
n=scan(n=1)
i=1
cat("The divisors of the number: ",n,"are shown below:")
for(i in 1:as.integer(n/2)){
if(n%%i==0)
print(i)
}
print(n)
```

Illustrative example-4.4 in R using iteration

Example-4.4. A number is called a Perfect Number if the sum of its divisors except itself equals the number. It is required to develop an R script for testing a given number as to whether it is a perfect number or not.

Task Analysis. We have already seen how to find out the divisors of a given number. So, the solution of this problem becomes a very easy one in the light of that concept. The only additional task we shall be required to do is obtaining the sum of the divisors and then to compare the sum with the given number to draw the required conclusion.

The requisite R script is shown below:

```
#Testing for a perfect number
cat("Enter a whole number to be tested")
N=scan(n=1)
sum=0
for(i in 1:as.numeric(N/2)){
if(N%%i==0)
sum=sum+i}
if(N==sum){
print("The number is a PERFECT NUMBER")
} else {
print("It is not a Perfect Number") }
```

Illustrative example-4.5 in R using iteration

Example-4.5. Narcissistic Number. if an n-digit number is equal to the sum of the th powers of its digits, it is called an n-narcissistic number. It is also sometimes known as an Armstrong number. For example;153, 1634, 558834 are such numbers.

It is required to develop an R script for checking whether a given number is a Narcissistic number or not.

Task Analysis: Here the first task is to find out the number of digits in the number. Next, we separate the digits from the number one by one, take the exponential value of the digit with the power equal to the number of digits in the number and sum up the values. At the end, we compare the sum of the exponential values of the digits with the given number and conclude.

The desired R script has been shown below:

Here, we have used the **paste() function** of R for string concatenation. In the function, **sep** stands for separator and the **collapse=NULL** implies that the concatenated value will have to be stored as a single value without unnecessary spaces.

Illustrative example-4.5 in R using iteration–Contd.

```
# Testing for Narcissistic property of a given number
cat("Enter any number for the testing")
N=scan(n=1)
M=N # We save here the given number for later reference
d=0
while(N>0){
d=d+1
N=N%/%10 # Integer division
}
string=""
tot=0
N=M
t=d
while(N>0){
r=N%%10
tot=tot+r^d
# The following 5 lines of code may be omitted by a novice programmer. It is to show: digits raised to the desired power sum up to give the given number
t=t-1
if(t!=0){
string=paste(string,paste(r,"^",d,"+ "),sep="",collapse=NULL) # This is for concatenation
} else {
string=paste(string,paste(r,"^",d),sep="",collapse=NULL) # This is also for concatenation without the  plus sign for the last digit } N=N%/%10
}
if(M==tot){
print("It is a Narcissistic Number")
cat("Because",M,"=",string)
} else {
print(" It is not a Narcissistic Number")
}
print("Thank You")
```

Illustrative example-4.6 in R using iteration

Example-4. 6. A sequence of numbers is said to follow the Fibonacci series, if the first three terms are 0, 1, 1 and the subsequent terms of the series can be obtained by taking the sum of the preceding two terms. It is required to develop an R script to generate any desired number of terms of the series.

Task Analysis: The finding out of the terms of the Fibonacci series is a very common programming task. We usually perform the task by initializing two variables with the first two terms which are known to us and then take the sum of their values to store in a third variable to have the third term. Now, after printing the value of the first term, we store the value of the second term in the variable containing the first term and that of the third term into the variable containing the second term. This will put the value of the second term in the first variable. So, we go on printing the value of the first variable until the desired number of terms has been printed.

The required R script has been shown below:

Illustrative example-4.6 in R using iteration–Contd.

```
# Program to print the numbers of Fibonacci series cat(" How many terms of the Fibonacci series would you like to see?")
n=scan(n=1)
t1=0
t2=1
print(" The Fibonacci terms are as shown below:")
for(i in 1:n){
print(t1)
t=t1+t2
t1=t2
t2=t
}
print(" End the program.... Thank You")
```

Illustrative example-4.7 in R using iteration

Example-4.7. It is required to test whether a given number is a prime number or not.

Task Analysis: We know that a whole number is said to be a **prime number** if it has no other divisor or factor except itself and unity (1). As 1 has no other factor except itself, it is excluded from the list of prime numbers. So, we are required to test from 2 to the given number to determine whether it has any other factor except 1 and itself **since 1 and the number itself are the common factors of all numbers, we need to test for other factors,if any**. Also, **we know that if a number has any factor at all, then at least one of the factors must lie within integer part of the square root of the number**. So, we can confine our checking from 2 to the integer part of the positive square root of the given number. If we do not get any factor or divisor within that range, we can safely conclude that the given number is a prime number. R provides us with **sqrt() function** to find out the square root of any real number. We shall make use of the function.

The required R script is shown below:

R script(Example-4.7) for testing the primality of a number

```
# Testing for primality of a given number
cat(" Enter the number to be tested for primality")
n=scan(n=1)
flag=0
sr=as.integer(sqrt(n))
i=2
while(i<= sr){
if(n%%i==0){
flag=1
break
}
i=i+1}
if(flag==0){
print(" The given number is a prime number")
} else {
print(" The given number is not a prime number")}
print(" End of program")
```

Illustrative example-4.8 in R using iteration

Example-4.8. It is required to develop an R script to determine the factorial value of a number.

Task Analysis: We know that the factorial value of a number is equal to the product of the integers from 1 to that number. So, the task is very easy and the R script for the task is shown below:

```
# Program to find out the factorial of a number
cat("Enter the number the factorial of which is required")
n=scan(n=1)
fact=1 # variable where we shall accumulate the factorial value
for(i in 1:n){
fact=fact*i
}
cat(" The factorial value of ", n,"is: ", fact)
print(" End of the Program.....Thank you!")
```

The concept of nested loop and Illustrative example-4.9 in R using iteration

Nested loop.

A nested loop is nothing but a loop within a loop. In order to solve some problems, we need to constructed nested loops. It is somewhat like nested if statements. The nesting can be done up to any depth. However, too much of nesting makes the program logic a bit complicated.

Let us see some illustrative problems in which nested loops will be required.

Example-4. 9. Let it be required to find out all the perfect numbers within 10000.

Task Analysis: The logic of finding out the perfect numbers has already been discussed earlier in this lesson. Now in order to find out all such perfect numbers within 10000,we need not accept any input, rather, we are required to check all numbers within 10000. Since, it is known that the first perfect number is 6, so, we start our checking all the natural numbers within 10000, starting from 6.

The required program is shown below:

The R code of the illustrative example-4.9

```
# Program to find out all the perfect numbers within 10000
for(i in 6:10000){
tot=0
for(j in 1:as.integer(i/2)){
if(i%%j==0)
tot=tot+j}
if(i==tot)
print(i)}
```

It may be observed that the outer loop has been used to generate the numbers one by one automatically while the inner loop has been used to find out all the divisors of the number under consideration and hence their sum.

Example-4.10. Let us find out all the Armstrong numbers within 9999.

Task Analysis: We have already seen the logic of finding out Armstrong numbers or Narcissistic Numbers earlier in this chapter. In that example, we tested a given input. Now, we are required to find out all such numbers within 9999.

The R code of the illustrative example-4.10

So, here ,we have to check all natural numbers from 1 to 9999. This checking will, therefore, require a nested loop. The outer loop for generating numbers to be checked and the inner loop for testing each of them.

The following is the required the code in R:

```
# Testing for Narcissistic property of a given number
cat("ALL the ARMSTRONG numbers within 9999")
for(N in 1:9999){
M=N # We save here the given number for later reference
d=0
while(N>0){
d=d+1
N=N%/%10 # Integer division}}
string=""
tot=0
N=M
t=d
while(N>0){
r=N%%10
tot=tot+r^d
t=t-1
if(t!=0){
string=paste(string,paste(r,"^",d,"+ "),sep="",collapse=NULL)
} else {
string=paste(string,paste(r,"^",d),sep="",collapse=NULL)}
N=N%/%1}
if(M==tot){
print(M)
print(paste("Because",M,"=",string))
}
}
```

Illustrative example-4.11 in R using iteration

Example-4.11. Let us now develop an R script for finding out all the prime numbers within 1000.

Task Analysis. Testing a given number for its primality has already been discussed and illustrated in an earlier problem of this chapter. In that problem, we tested a given number. Here, we are required to check all the numbers within 1000. So, inputting is not necessary here; rather, we shall take all the natural numbers from 2 to 1000 automatically through a loop to check for the primality. Hence, we require a nested loop for the checking by generating natural numbers from 2 to 1000. The following R-code shows requisite program:

```
for(n in 2:1000){ # Testing for primality of a given number
flag=0
sr=as.integer(sqrt(n))
i=2
while(i<= sr){
if(n%%i==0){
flag=1
break}
i=i+1}
if(flag==0){
print(n) } }
```

Illustrative example-4.12 in R using iteration

Example-12. Some three-digit numbers show the property that the sum of the factorials of the digits equals the numbers. For example; 145=1!+4!+5!
Let us develop a program in R to find out all such numbers within 9999999.

```
# Program to determine the numbers the sum of the factorials of the digits
of which equal the number.
for(n in 1:9999999)
{s=0
m=n
while(m>0){
r=m%%10
fact=1
while(r>1)
{fact=fact*r
r=r-1}
s=s+fact
m=m%/%10
}
if(s==n)
print(n)
}
```

Illustrative example-4.13 in R using iteration

Example-4.13.Some two-digit numbers have the property that the sum of the squares of the numbers equals the sum of the squares of the numbers with reversed digits. For example:
$48^2 + 52^2 + 63^2 = 84^2 + 25^2 + 36^2$
Let us develop a program to find out all such two-digit triplets.
Task Analysis: Here, we shall have to check all possible triplets of two-digit numbers .The following coding in R gives the desired output:

```
#Sum of squares of numbers equaling the sum of squares of the numbers formed with reversed digits
for(i in 10:99){
for(j in (i+1):98){
 for(k in (j+1):97)
{if(j>98|| k>97)
next
 n1=i^2
 n2=j^2
n3=k^2
 r1=i%%10
 r2=i%/%10
 rn1=r1*10+r2
 r1=j%%10
r2=j%/%10
 rn2=r1*10+r2
r1=k%%10
r2=k%/%10
 rn3=r1*10+r2
 rn11=rn1^2
 rn22=rn2^2
 rn33=rn3^2
 str1=""
 str2=""
if((n1+n2+n3)==(rn11+rn22+rn33)){
str1=paste(str1,as.character(i),"^2+",as.character(j),"^2+",as.character(k),"^2",sep="",collapse=NULL)
 str2= paste(str2,as.character(rn1),"^2+",as.character(rn2),"^2+",as.character(rn3),"^2",sep="",collapse=NULL)
 print(paste(str1,"=",str2))
} } }
```

Illustrative example-4.14 in R using iteration

Example-4.14. Given the mathematical series:
x+ $\frac{x^2}{2}$ +$\frac{x^3}{3}$ +.....up to n terms.
It is required to develop an R-script to obtain sum of the series for for sets of values of x and n.

Task Analysis: The task is very easy for a novice programmer also because the power sign is supported by R. But, we should not go on evaluating such exponential expressions; because for very large n, the computation will have a high time complexity. We may rather use an intermediate variable, say t, for evaluating different exponential forms of x through a simple multiplication of the value of x in the preceding term by another x. To make further clear, we show below the computational part for evaluation of different exponential forms of x.

```
 Input x,n
 Let t=1; sum=0
 For each I from 1 to n Do
 t=t*x
sum=sum+t/i
Endfor
```

The R code of the illustrative example-4.14

The following R code serves the purpose:

```
# Sum of Series
repeat{
cat(" Enter the value of x followed by number of terms to be added")
v=scan(n=2)
x=v[1]
n=v[2]
sum=0
t=1
for(i in 1:n){
t=t*x
sum=sum+t/i}
cat(" The sum of the series for x=",x,"and n=",n,"is:",sum)
ch=readline(prompt="Would you like to continue?(Y/y) for 'yes' or (Y/y)
for 'No'")
if(ch=="y"||ch=="Y"){
print(" Please carry on")
} else {
print("Thank you for running the program")
break}}
```

Illustrative example-4.15 in R using iteration

Example-4.15. It is required to develop an R script to determine the sum of the following series for different sets of values of x and n:

x-$\frac{x^2}{2!}$ +$\frac{x^3}{3!}$ - $\frac{x^5}{5!}$ $\pm$ ..... Up to n terms

Task Analysis: Here also, we shall follow the logic similar to the above solution. This we shall require not only the exponential expressions in x, but also constant coefficients in factorial form. Further, there is a change of sign in alternate terms. The basic part of the algorithm may be as shown below:

Input x,n
Sum=0
Term=-1
For each i from 1 to n do
Term=-Term*x/i
Sum=Sum+Term
Endfor

It may be noted that the statement for the evaluation of Term determines each term with appropriate sign and factorial value. No additional calculation is needed for the determination of the factorials. This is, therefore, the optimum way of determining the sum of the series.

The R script for evaluating such a series is shown below:

The R code of the illustrative example-4.15

```
 # series2
repeat{
cat("Enter the value of x followed by the value of number of terms n")
v=scan(n=2)
x=v[1]
n=v[2]
term=-1
sum=0
for(i in 1:n){
term=-term*x/i
sum=sum+term}
cat(" The sum of the series for x=",x,"and n=",n,"is:",sum)
ch=readline(prompt="Would you like to continue?(Y/y) for 'yes' or (Y/y)
for 'No'")
 if(ch=="y"||ch=="Y"){
 print(" Please carry on")
} else {
print("Thank you for running the program")
 break } }
```

Illustrative example-4.16 in R using iteration

Example-4.16. It is known that a contractor pays his workers at the end of each week on the basis of the total hours worked in the week according to the following rules: For the first 35 hours, the rate of pay is fixed at Rs,19 per hour; for the next 25 hours, the rate is Rs. 29 per hour and for the rest, if any, the rate is fixed at Rs, 37 per hour. Moreover, no worker is permitted to work for more than 90 hours in a week. It is required to devise a program in R to compute the weekly wage of the workers.The contractor demands to know total amount paid and the total number of workers paid at the of all payments.

Task Analysis: Clearly, the only input requirement of the program to be written is the total hours worked by a worker in a week. Having accepted the input, we need to check whether the given input exceeds 80; if so, we terminate the program; otherwise, we check the different conditions as per the stated rules and calculate the wage accordingly.Moreover, we would require setting up an indefinite loop so that we can repeat the process any number of times. When we signal the end of input, the system would show the totals. The following is the requisite R Code:

The R code of the illustrative example-4.16

```
# This is to calculate the wage of the workers under a contractor as per stated rules.
tot=0
cnt=0
repeat{
cat("Enter the hours worked in a week")
h=scan(n=1);
if (h>80){
print("Invalid Hours")
print("Try again from the beginning with a valid hours of work done which shoud be <= 80")
next
} else if(h<=35){
w=h*19
} else if(h<=60){
w=35*19+(h-35)*29
} else {
w=35*19+25*29+(h-60)*37
}
cat("The wage for this worker will be Rs. :",w,"for ",h,"hours of work")
tot=tot+w
cnt=cnt+1
ch=readline(prompt="continue?(y/n)")
if(ch=="n"||ch=="N"){
break
}
}
cat("The total wage paid is Rs.: ",tot, "and the total number of workers paid is: ", cnt)
print("End of R script.... Thank You")
```

Illustrative example-4.17 in R using iteration

Example-4.17. It is required to develop a program to obtain **GCD(HCF)** and **LCM** of two given numbers.

Task analysis: Here, we adopt the Euclid's basic formula for the evaluation by using the Division-Remainder method. A number is said to be a **GCD** of another number if it can divide the latter without leaving any remainder; otherwise, we find out the greatest number that can divide both the numbers without leaving any remainder. The procedure that we follow here is that we divide the first number with the second one and obtain the remainder; if the remainder is zero, we stop immediately and conclude that the last divisor is the **GCD** of both; otherwise, we store the last divisor in the place of the last dividend and then we store the last remainder in the place of the divisor and then check the remainder again; we continue the process until we get a number that leaves zero remainder. The divisor that leaves zero remainder is the desired **GCD**. To obtain the **LCM** of the numbers, we utilize the formula:

LCM X HCF of two numbers= Product of the numbers.

This method has been shown in the R-script below:

The R code of the illustrative example-4.17

```
# Finding out the HCF and LCM of two numbers
cat("Enter any two numbers of your choice")
m=scan(n=2)
a=as.integer(m[1]);b=as.integer(m[2])
p=a*b
repeat{
r=a%%b;a=b;b=r
if(r==0)
break
}
l=p/a;print(l)
cat("The HCF of the numbers is: ",a)
cat("And the LCM is: ",l)
print("End of program....")
```

Illustrative example-4.18 in R using iteration

Example-4.18. A number is said to be **a triad number** if the double and triple of the number contain all separate digits with no repetition of any of them. It is required to print out all triad numbers within 10000.

Task Analysis: Here our checking should start from 1 and to be continued up to 10000. For each of the digits of the doubled number, we are required to compare with each of the digits of the tripled number, If no repetition is found among the digits, we conclude that the number under consideration is a triad number. The following is the required R code for the testing and printing of the triad numbers.

```
# Checking for triad numbers within 10000
for (i in 1:10000){
d=2*i
t=3*i
while(d>0){
x=d%%10
d=d%/%10
flag=0
dt=t
while(dt>0){ y=dt%%10
if(x==y){
flag=1
break}
dt=dt%/%10
}
if(flag==1)
break}
if(flag==0){
cat(i,"is a triad Number")
ch=readline(prompt="Press any key to continue")}
```

Summary of this lesson:

Here, we have learnt different types of **loop structures** that R supports. **R supports three loop structures, namely, 'while loop',' for loop' and 'repeat loop'.** The purpose of all types of loops is the same; they differ in their syntax structures and the nature of execution.
We have also studied a large number of illustrative examples implementing each loop structure. To explore the arena of 'looping', every reader should go through the solutions of a large range of problems from very simple to very hard, before stepping forward to solve the problems in the exercises.